ПРИЛОЖЕНИЕ А
(обязательное)

Листинг кода

EventParticipantsManagementViewModel.swift

```
final class EventParticipantsManagementViewModel: ObservableObject {
    @Published private(set) var applications: [EventParticipantResponse]
= []
    @Published private(set) var isLoading = false
    @Published var errorMessage: String?

    private let eventID: String
    private let session: AppSession
    private let api: EventAPIProtocol

    init(eventID: String, session: AppSession, api: EventAPIProtocol? =
nil) {
        self.eventID = eventID
        self.session = session
        self.api = api ?? EventAPI(baseURL: URL(string:
AppConfig.baseURLString)!)
    }

    var participants: [EventParticipantResponse] {
        applications.filter {
            $0.applicationStatus == .accepted || $0.applicationStatus ==
.pending
        }
    }

    var applicationRequests: [EventParticipantResponse] {
        applications.filter { application in
            !application.isCreator && application.applicationStatus !=
.cancelled
        }
    }

    func loadApplications() async {
        isLoading = true
        errorMessage = nil
        defer { isLoading = false }

        do {
            applications = try await session.performAuthorizedRequest {
token in
                try await api.fetchEventApplications(eventID: eventID,
token: token)
            }
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    func removeParticipant(_ participant: EventParticipantResponse,
reason: String) async {
        let normalizedReason = reason.trimmingCharacters(in:
.whitespacesAndNewlines)
        guard !normalizedReason.isEmpty else {
            errorMessage = "Укажите причину удаления"
            return
```

```
        }

        isLoading = true
        errorMessage = nil
        defer { isLoading = false }

        do {
            _ = try await session.performAuthorizedRequest { token in
                try await api.removeParticipant(
                    applicationID: participant.applicationID,
                    reason: normalizedReason,
                    token: token
                )
            }
            await loadApplications()
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    func acceptApplication(_ application: EventParticipantResponse)
async {
        await reviewApplication(application, accept: true)
    }

    func rejectApplication(_ application: EventParticipantResponse)
async {
        await reviewApplication(application, accept: false)
    }

    private func reviewApplication(_ application:
EventParticipantResponse, accept: Bool) async {
        isLoading = true
        errorMessage = nil
        defer { isLoading = false }

        do {
            _ = try await session.performAuthorizedRequest { token in
                if accept {
                    try await api.acceptApplication(id:
application.applicationID, token: token)
                } else {
                    try await api.rejectApplication(id:
application.applicationID, token: token)
                }
            }
            await loadApplications()
        } catch {
            errorMessage = error.localizedDescription
        }
    }
}

    private func loadData() async {
        isLoading = true
        defer { isLoading = false }

        do {
            if !isRestoring {
                _ = try await session.performAuthorizedRequest { token
in
                    try await api.startEvent(id: eventID, token: token)
                }
            }
```

```
            let items = try await session.performAuthorizedRequest {
token in
                try await api.fetchAttendance(eventID: eventID, token:
token)
            }
            attendanceItems = items
            let serverPresent = Set(items.filter { $0.isPresent }.map {
$0.applicationID })
            // If no one is marked present yet (fresh start), default to
everyone present
            presentIDs = serverPresent.isEmpty ? Set(items.map {
$0.applicationID }) : serverPresent
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    private func confirmAttendance() async {
        isConfirming = true
        defer { isConfirming = false }

        do {
            let updatedEvent = try await
session.performAuthorizedRequest { token in
                try await api.confirmAttendance(
                    eventID: eventID,
                    presentApplicationIDs: Array(presentIDs),
                    token: token
                )
            }

            if updatedEvent.status?.lowercased() == "grouping" {
                withAnimation(.easeInOut(duration: 0.25)) {
                    groupingEvent = updatedEvent
                }
            } else {
                onConfirmed?(updatedEvent)
                dismiss()
            }
        } catch {

    private func loadAllGroups() async {
        do {
            let fetched = try await session.performAuthorizedRequest {
token in
                try await api.fetchGroups(eventID: eventID, token:
token)
            }
            allGroups = fetched
            if let idx = fetched.firstIndex(where: { $0.id ==
initialGroup.id }) {
                currentGroupIndex = idx
            }
        } catch { }
    }

    private func updateGroupState(_ updated: EventGroup) {
        if let index = allGroups.firstIndex(where: { $0.id == updated.id
}) {
            allGroups[index] = updated
        }
    }
```

```
    private func addGroup() async {
        isAdding = true
        defer { isAdding = false }
        do {
            let newGroup = try await session.performAuthorizedRequest {
token in
                try await api.addGroup(eventID: eventID, token: token)
            }
            allGroups.append(newGroup)
            withAnimation(.easeInOut(duration: 0.2)) {
                currentGroupIndex = allGroups.count - 1
            }
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    private func deleteCurrentGroup() async {
        guard let group = currentGroup else { return }
        isDeleting = true
        defer { isDeleting = false }
        do {
            try await session.performAuthorizedRequest { token in
                try await api.deleteGroup(eventID: eventID, groupNumber:
group.groupNumber, token: token)
            }
            let fetched = try await session.performAuthorizedRequest {
token in
                try await api.fetchGroups(eventID: eventID, token:
token)
            }
            if fetched.isEmpty {
                onGoToList()
            } else {
                allGroups = fetched

    private func setLeader(profileID: Int) async {
        guard let group = currentGroup else { return }
        do {
            let updated = try await session.performAuthorizedRequest {
token in
                try await api.setGroupLeader(
                    eventID: eventID,
                    groupNumber: group.groupNumber,
                    profileID: profileID,
                    token: token
                )
            }
            updateGroupState(updated)
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    private func removeLeader() async {
        guard let group = currentGroup else { return }
        do {
            let updated = try await session.performAuthorizedRequest {
token in
                try await api.removeGroupLeader(
                    eventID: eventID,
                    groupNumber: group.groupNumber,
                    token: token
                )
```

```
            }
            updateGroupState(updated)
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    private func addMember(profileID: Int) async {
        guard let group = currentGroup else { return }
        do {
            let updated = try await session.performAuthorizedRequest {
token in
                try await api.addGroupMember(
                    eventID: eventID,
                    groupNumber: group.groupNumber,
                    profileID: profileID,
                    token: token
                )
            }
            updateGroupState(updated)
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    private func removeMember(profileID: Int) async {
        guard let group = currentGroup else { return }
        do {
            let updated = try await session.performAuthorizedRequest {
token in
                try await api.removeGroupMember(
                    eventID: eventID,
                    groupNumber: group.groupNumber,
                    profileID: profileID,
                    token: token
                )
            }
            updateGroupState(updated)
        } catch {

    private func applyAutoFill() async {
        guard let group = currentGroup else { return }
        isAutoFilling = true
        defer { isAutoFilling = false }
        let toAdd = Array(unassignedProfiles.prefix(autoFillCount))
        var current = group
        for profile in toAdd {
            do {
                current = try await session.performAuthorizedRequest {
token in
                    try await api.addGroupMember(eventID: eventID,
groupNumber: group.groupNumber,
                        profileID: profile.id, token: token)
                }
            } catch {
                errorMessage = error.localizedDescription
                break
            }
        }
        updateGroupState(current)
    }

    private func confirmAllGroups() async {
        isConfirming = true
```

```
        defer { isConfirming = false }
        do {
            let updatedEvent = try await
session.performAuthorizedRequest { token in
                try await api.confirmGroups(eventID: eventID, token:
token)
            }
            onDone(updatedEvent)
        } catch {
            errorMessage = error.localizedDescription
        }
    }
```

EventViewModel.swift

```
import Foundation
import CoreLocation
import Combine
import UIKit

@MainActor
final class EventViewModel: ObservableObject {
    @Published var eventTitle = ""
    @Published var selectedDirection = ""
    @Published var eventDescription = ""

    @Published var locationText = ""
    @Published var selectedCoordinate: CLLocationCoordinate2D?

    @Published var startDate: Date?
    @Published var startTime: Date?

    @Published var endDate: Date?
    @Published var endTime: Date?

    @Published var volunteersCount: Int = 1
    @Published var volunteersManualInput: String = "1"
    @Published var eventPhotoImage: UIImage?
    @Published var organizerName = ""
    @Published var isOrganizerVerified = false
    @Published var isVerifiedOrganization = false
    @Published var ratingPoints = 50

    @Published var isSubmitting = false
    @Published var isProfileContextLoading = false
    @Published var errorMessage: String?
    @Published var successMessage: String?
    @Published private(set) var locationContext: EventLocationContext?

    private let session: AppSession
    private let profileAPI: ProfileAPIProtocol
    private let eventAPI: EventAPIProtocol
    private let geocodingAPI: GeocodingAPIProtocol

    private var selectedLocation: EventLocationSelection?
    private var eventPhotoData: Data?
    private var eventPhotoURL: String?
    private var hasLoadedProfileContext = false

    init(
        session: AppSession,
        profileAPI: ProfileAPIProtocol? = nil,
        eventAPI: EventAPIProtocol? = nil,
        geocodingAPI: GeocodingAPIProtocol? = nil
```

```
    ) {
        self.session = session
        self.profileAPI = profileAPI ?? ProfileAPI(baseURL: URL(string:
AppConfig.baseURLString)!)
        self.eventAPI = eventAPI ?? EventAPI(baseURL: URL(string:
AppConfig.baseURLString)!)
        self.geocodingAPI = geocodingAPI ?? GeocodingAPI(baseURL:
URL(string: AppConfig.baseURLString)!)
    }

    var todayStart: Date {
        Calendar.current.startOfDay(for: Date())
    }

    var canSubmit: Bool {
        !trim(eventTitle).isEmpty &&
        !trim(selectedDirection).isEmpty &&
        !trim(eventDescription).isEmpty &&
        selectedCoordinate != nil &&
        !trim(locationText).isEmpty &&
        startDate != nil &&
        startTime != nil &&
        !trim(volunteersManualInput).isEmpty &&
        volunteersInputIsValid &&
        endError == nil
    }

    func loadProfileContextIfNeeded() async {
        guard !hasLoadedProfileContext, !isProfileContextLoading else {
return }

        isProfileContextLoading = true
        defer { isProfileContextLoading = false }

        do {
            let profile = try await session.performAuthorizedRequest {
token in
                try await profileAPI.fetchMyProfile(token: token)
            }
            let country = canonicalCountry(from: profile.country)
            let city = canonicalCity(from: profile.city, in: country)
            organizerName = displayName(from: profile)
            isOrganizerVerified = profile.isVerified ?? false
            isVerifiedOrganization = (profile.type == "organization") &&
(profile.isVerified ?? false)
            let searchArea = CityDirectory.searchArea(for: country)
            let cityCoordinate = try await
geocodingAPI.geocodeCity(city: city, country: country, area: searchArea)
                ?? searchArea.center

            locationContext = EventLocationContext(
                country: country,
                city: city,
                countrySearchArea: searchArea,
                cityCoordinate: cityCoordinate
            )
            hasLoadedProfileContext = true
        } catch {
            let country = CityDirectory.defaultCountry
            let city = CityDirectory.cities(for: country).first ??
"Минск"
            let searchArea = CityDirectory.searchArea(for: country)

            locationContext = EventLocationContext(
```

```
                country: country,
                city: city,
                countrySearchArea: searchArea,
                cityCoordinate: searchArea.center
            )
            hasLoadedProfileContext = true
        }
    }

    func submit() async -> EventResponse? {
        clearMessages()

        guard canSubmit else {
            errorMessage = "Заполните все обязательные поля"
            return nil
        }

        guard let selectedCoordinate,
              let country = selectedLocation?.country ??
locationContext?.country,
              let city = selectedLocation?.city ??
locationContext?.city,
              !country.isEmpty,
              !city.isEmpty else {
            errorMessage = "Не удалось определить местоположение
события"
            return nil
        }

        guard let volunteersNeeded = Int(volunteersManualInput),
              let startDate,
              let startTime else {
            errorMessage = "Заполните все обязательные поля"
            return nil
        }

        isSubmitting = true
        defer { isSubmitting = false }

        do {
            var uploadedPhotoURL = eventPhotoURL
            if let eventPhotoData {
                uploadedPhotoURL = try await
session.performAuthorizedRequest { token in
                    try await eventAPI.uploadEventPhoto(data:
eventPhotoData, token: token)
                }
            }

            let request = CreateEventRequest(
                title: trim(eventTitle),
                direction: trim(selectedDirection),
                description: trim(eventDescription),
                country: CityDirectory.canonicalCountryName(for:
country),
                city: city,
                locationName: trim(locationText),
                photoURL: uploadedPhotoURL,
                latitude: selectedCoordinate.latitude,
                longitude: selectedCoordinate.longitude,
                startsAt: iso8601String(from: combine(day: startDate,
time: startTime)),
                endsAt: endTimestamp,
                volunteersNeeded: volunteersNeeded,
```

```
                ratingPoints: isVerifiedOrganization ? ratingPoints :
nil
            )

            let response = try await session.performAuthorizedRequest {
token in
                try await eventAPI.createEvent(request, token: token)
            }
            successMessage = response.message ?? "Событие создано"
            eventPhotoURL = uploadedPhotoURL
            eventPhotoData = nil
            resetFormKeepingContext()
            return response
        } catch {
            errorMessage = error.localizedDescription
            return nil
        }
    }
```

MyEventsViewModel.swift

```
import Foundation
import Combine

@MainActor
final class MyEventsViewModel: ObservableObject {
    @Published private(set) var events: [EventResponse] = []
    @Published private(set) var isLoading = false
    @Published var errorMessage: String?

    private let session: AppSession
    private let api: EventAPIProtocol
    private var currentLoadID: UUID?

    init(
        session: AppSession,
        api: EventAPIProtocol? = nil
    ) {
        self.session = session
        self.api = api ?? EventAPI(baseURL: URL(string:
AppConfig.baseURLString)!)
    }

    var isEmpty: Bool {
        events.isEmpty && !isLoading
    }

    func loadMyEvents(filter: MyEventsFilter? = nil) async {
        let loadID = UUID()
        currentLoadID = loadID

        isLoading = true
        errorMessage = nil
        defer {
            if currentLoadID == loadID {
                isLoading = false
            }
        }

        do {
            let loadedEvents = try await
session.performAuthorizedRequest { token in
                try await api.fetchMyEvents(filter: filter, token:
token)
```

```
            }
            guard currentLoadID == loadID else { return }
            events = loadedEvents
        } catch {
            guard currentLoadID == loadID else { return }
            errorMessage = error.localizedDescription
        }
    }

    func prependOrUpdate(_ event: EventResponse) {
        if let index = events.firstIndex(where: { $0.id == event.id }) {
            events[index] = event
        } else {
            events.insert(event, at: 0)
        }
    }
}
```

EventAPI.swift

```
func createEvent(_ request: CreateEventRequest, token: String?) async
throws -> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events"))
      urlRequest.httpMethod = "POST"
      urlRequest.setValue("application/json", forHTTPHeaderField:
"Content-Type")

    if let token, !token.isEmpty {
        urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
}

urlRequest.httpBody = try JSONEncoder().encode(request)
let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
try validate(response: response, data: data)
return try JSONDecoder().decode(EventResponse.self, from: data)

func fetchEvent(id: String, token: String?) async throws ->
EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(id)"))
    urlRequest.httpMethod = "GET"
    if let token, !token.isEmpty {
        urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
}

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode(EventResponse.self, from: data)

func deleteEvent(id: String, token: String) async throws {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(id)"))
    urlRequest.httpMethod = "DELETE"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
```

```
}

func startEvent(id: String, token: String) async throws -> EventResponse
{
    try await postEventAction(path: "api/events/\(id)/start", token:
token)
}

func cancelEvent(id: String, reason: String, token: String) async throws
-> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(id)/cancel"))
    urlRequest.httpMethod = "POST"
    urlRequest.setValue("application/json", forHTTPHeaderField:
"Content-Type")
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    urlRequest.httpBody = try JSONEncoder().encode(["reason": reason])

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)

func fetchEventFeed(searchText: String?, token: String?) async throws ->
[EventResponse] {
    try await fetchEventFeed(searchText: searchText, filters: .empty,
token: token)
}

func fetchEventFeed(searchText: String?, filters: EventFeedFilters,
token: String?) async throws -> [EventResponse] {
    var components = URLComponents(
    url: baseURL.appendingPathComponent("api/events/feed"),
    resolvingAgainstBaseURL: false
    )

    var queryItems: [URLQueryItem] = []
    let query = searchText?.trimmingCharacters(in:
.whitespacesAndNewlines) ?? ""
    if !query.isEmpty {
        queryItems.append(URLQueryItem(name: "q", value: query))
    }

    if let direction = filters.direction?.trimmingCharacters(in:
.whitespacesAndNewlines),
    !direction.isEmpty {
    queryItems.append(URLQueryItem(name: "direction", value: direction))
}

func finishEvent(id: String, token: String) async throws ->
EventResponse {
    try await postEventAction(path: "api/events/\(id)/finish", token:
token)
}

func fetchRatableProfiles(eventID: String, token: String) async throws -
> [RatableProfile] {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/ratings"))
    urlRequest.httpMethod = "GET"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
```

```
    try validate(response: response, data: data)
    return try JSONDecoder().decode([RatableProfile].self, from: data)
}

func submitRatings(eventID: String, ratings: [RatingItem], token:
String) async throws -> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/ratings"))
    urlRequest.httpMethod = "POST"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    urlRequest.setValue("application/json", forHTTPHeaderField:
"Content-Type")
    urlRequest.httpBody = try
JSONEncoder().encode(SubmitRatingsRequest(ratings: ratings))
    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode(EventResponse.self, from: data)
}

private func validate(response: URLResponse, data: Data) throws {
    guard let httpResponse = response as? HTTPURLResponse else {
        throw EventAPIError.invalidResponse
    }

    if httpResponse.statusCode == 401 {
        throw EventAPIError.unauthorized
    }

    guard (200...299).contains(httpResponse.statusCode) else {
        let message = (try? JSONDecoder().decode(APIErrorMessage.self,
from: data).message)
?? HTTPURLResponse.localizedString(forStatusCode:
httpResponse.statusCode)
        throw EventAPIError.server(message)
    }
}

private func postEventAction(path: String, token: String) async throws -
> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent(path))
    urlRequest.httpMethod = "POST"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
}
```

EventParticipantsAPI.swift

```
func applyToEvent(id: String, token: String) async throws ->
EventResponse {
    try await postEventAction(path: "api/events/\(id)/applications",
token: token)
}

func cancelApplication(eventID: String, token: String) async throws ->
EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/applications"))
```

```
    urlRequest.httpMethod = "DELETE"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode(EventResponse.self, from: data)
}
func acceptApplication(id: Int, token: String) async throws ->
EventResponse {
    try await postEventAction(path:
"api/events/applications/\(id)/accept", token: token)
}

func rejectApplication(id: Int, token: String) async throws ->
EventResponse {
    try await postEventAction(path:
"api/events/applications/\(id)/reject", token: token)
}

func fetchEventApplications(eventID: String, token: String) async throws
-> [EventParticipantResponse] {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/applications"))
    urlRequest.httpMethod = "GET"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode([EventParticipantResponse].self,
from: data)
}

func removeParticipant(applicationID: Int, reason: String, token:
String) async throws -> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/applications/\(applicationID)
/remove"))
    urlRequest.httpMethod = "POST"
    urlRequest.setValue("application/json", forHTTPHeaderField:
"Content-Type")
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    urlRequest.httpBody = try
JSONEncoder().encode(RemoveEventParticipantRequest(reason: reason))
    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode(EventResponse.self, from: data)
}

func fetchAttendance(eventID: String, token: String) async throws ->
[AttendanceItem] {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/attendance"))
    urlRequest.httpMethod = "GET"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
```

```
    try validate(response: response, data: data)
    return try JSONDecoder().decode([AttendanceItem].self, from: data)
}

func confirmAttendance(eventID: String, presentApplicationIDs: [Int],
token: String) async throws -> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/attendance/confirm
"))
    urlRequest.httpMethod = "POST"
    urlRequest.setValue("application/json", forHTTPHeaderField:
"Content-Type")
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    urlRequest.httpBody = try
JSONEncoder().encode(ConfirmAttendanceRequest(presentApplicationIDs:
presentApplicationIDs))

    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode(EventResponse.self, from: data)
}

func fetchRatableProfiles(eventID: String, token: String) async throws -
> [RatableProfile] {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/ratings"))
    urlRequest.httpMethod = "GET"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode([RatableProfile].self, from: data)
}

func submitRatings(eventID: String, ratings: [RatingItem], token:
String) async throws -> EventResponse {
    var urlRequest = URLRequest(url:
baseURL.appendingPathComponent("api/events/\(eventID)/ratings"))
    urlRequest.httpMethod = "POST"
    urlRequest.setValue("Bearer \(token)", forHTTPHeaderField:
"Authorization")
    urlRequest.setValue("application/json", forHTTPHeaderField:
"Content-Type")
    urlRequest.httpBody = try
JSONEncoder().encode(SubmitRatingsRequest(ratings: ratings))
    let (data, response) = try await NetworkRequestExecutor.data(for:
urlRequest, session: session)
    try validate(response: response, data: data)
    return try JSONDecoder().decode(EventResponse.self, from: data)
}
```

EventRatingView.swift

```
struct EventRatingView: View {
    let eventID: String
    let session: AppSession
    let api: EventAPIProtocol
    let isOrganizerVerified: Bool
    let eventRatingPoints: Int
    var onCompleted: ((EventResponse?) -> Void)?
```

```
    private var submitButton: some View {
        Button {
            Task { await submitRatings() }
        } label: {
            ZStack {
                RoundedRectangle(cornerRadius: 14, style: .continuous)
                    .fill(navy)
                if isSubmitting {
                    ProgressView().tint(.white)
                } else {
                    Text("Отправить")
                        .font(.system(size: 16, weight: .semibold))
                        .foregroundColor(.white)
                }

    private func loadProfiles() async {
        isLoading = true
        defer { isLoading = false }
        do {
            let fetched = try await session.performAuthorizedRequest {
token in
                try await api.fetchRatableProfiles(eventID: eventID,
token: token)
            }
            profiles = fetched
            for p in profiles {
                selectedScores[p.profileID] = nil
            }
        } catch {
            errorMessage = error.localizedDescription
        }
    }

    private func submitRatings() async {
        isSubmitting = true
        defer { isSubmitting = false }
        do {
            let items = profiles.map { profile -> RatingItem in
                let raw: Int = (selectedScores[profile.profileID] ??
nil) ?? 0
                let score: Int
                if raw == eventRatingPoints && isOrganizerVerified {
                    // Star selected: add % bonus on top of event points
                    let bonusPct = profile.isLeader ? 0.30 : 0.20
                    let bonus = Int(Double(profile.currentRating) *
bonusPct)
                    score = raw + bonus
                } else {
                    score = raw
                }
                return RatingItem(profileID: profile.profileID, score:
score)
            }
            let updated = try await session.performAuthorizedRequest {
token in
                try await api.submitRatings(eventID: eventID, ratings:
items, token: token)
            }
            onCompleted?(updated)
        } catch {
            errorMessage = error.localizedDescription
        }
    }
```

```
}
```

EventService.py

```
def create_event(
        self,
        db: Session,
        payload: CreateEventRequest,
        user: User,
        background_tasks: BackgroundTasks | None = None,
    ) -> EventResponse:
        if not user.is_active:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="User is blocked")

        creator_profile = self._profile_for_user(db, user)
        direction = self._direction_by_name(db, payload.direction)

        is_verified_org = (
            creator_profile.type == ProfileType.organization
            and bool(creator_profile.is_verified)
        )
        rating_points = 50
        if is_verified_org and payload.rating_points is not None:
            rating_points = max(50, min(100, payload.rating_points))

        event = Event(
            creator_id=user.id,
            title=payload.title,
            description=payload.description,
            country=payload.country,
            city=payload.city,
            location_name=payload.location_name,
            photo_url=payload.photo_url,
            latitude=payload.latitude,
            longitude=payload.longitude,
            starts_at=payload.starts_at,
            ends_at=payload.ends_at,
            volunteers_needed=payload.volunteers_needed,
            rating_points=rating_points,
            status="pending",
        )
        event.directions.append(direction)

        db.add(event)
        db.flush()
        if creator_profile.type == ProfileType.volunteer:
            db.add(
                EventApplication(
                    event_id=event.id,
                    volunteer_id=creator_profile.id,
                    status=EventApplicationStatus.accepted,
                )
            )

        notification_service.create(
            db,
            user.id,
            "Событие отправлено для подтверждения, ожидайте.",
        )

        db.commit()
        db.refresh(event)
```

```
        if background_tasks is not None:
            background_tasks.add_task(
                telegram_admin_bot.notify_admins_about_event_id,
                event.id,
            )

        response = self._response(db, event, user)
        response.message = "Событие отправлено для подтверждения,
ожидайте."
        return response

    def start_event(self, db: Session, event_id: str, user: User) ->
EventResponse:
        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        if event.creator_id != user.id:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can start event")

        normalized_status = (event.status or "").strip().lower()
        if normalized_status not in ("approved", "active"):
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event cannot be started")

        if self._accepted_count(db, event_id) < event.volunteers_needed:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Not enough participants to start event",
            )

        creator_profile =
db.scalar(select(Profile).where(Profile.user_id == user.id))
        accepted_applications = db.scalars(
            select(EventApplication).where(
                EventApplication.event_id == event_id,
                EventApplication.status ==
EventApplicationStatus.accepted,
                EventApplication.volunteer_id != (creator_profile.id if
creator_profile else -1),
            )
        ).all()

        event.status = "attendance"
        db.flush()

        existing_attendance_ids = {
            row[0]
            for row in db.execute(

select(EventAttendance.application_id).where(EventAttendance.event_id ==
event_id)
            )
        }
        for application in accepted_applications:
            if application.id not in existing_attendance_ids:
                db.add(EventAttendance(event_id=event_id,
application_id=application.id, is_present=False))

        db.commit()
        db.refresh(event)
```

```
        response = self._response(db, event, user)
        response.message = "Отметьте присутствующих"
        return response

    def finish_event(self, db: Session, event_id: str, user: User) ->
EventResponse:
        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        if event.creator_id != user.id:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can finish event")

        if (event.status or "").strip().lower() != "active":
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Only active events can be finished")

        event.status = "rating"

        accepted_applications = db.scalars(
            select(EventApplication).where(
                EventApplication.event_id == event.id,
                EventApplication.status ==
EventApplicationStatus.accepted,
            )
        ).all()

        event_title = event.title.strip() or "событие"
        for application in accepted_applications:
            if application.volunteer.user_id != user.id:
                notification_service.create(
                    db,
                    application.volunteer.user_id,
                    f"Событие «{event_title}» завершено. Оцените
участников.",
                    event_id=event.id,
                )

        db.commit()
        db.refresh(event)
        response = self._response(db, event, user)
        response.message = "Оцените участников события"
        return response

    def _response(self, db: Session, event: Event, user: User | None =
None) -> EventResponse:
        response = EventResponse.model_validate(event)
        response.direction = self._event_direction_name(event)
        response.organizer_name = self._organizer_name(event.creator)
        response.accepted_count = self._accepted_count(db, event.id)
        response.is_creator = user is not None and event.creator_id ==
user.id
        response.user_application_status =
self._user_application_status(db, event.id, user)
        response.is_organizer_verified =
self._organizer_is_verified(event.creator)
        response.organizer_type = self._organizer_type(event.creator)
        return response

    @staticmethod
```

EventParticipantsService.py

```
def apply_to_event(self, db: Session, event_id: str, user: User) ->
EventResponse:
        if not user.is_active:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="User is blocked")

        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        profile = self._profile_for_user(db, user)
        accepted_count = self._accepted_count(db, event.id)
        existing_application = self._application_for_profile(db,
event.id, profile.id)

        if event.creator_id == user.id:
            if profile.type == ProfileType.organization:
                raise HTTPException(
                    status_code=status.HTTP_400_BAD_REQUEST,
                    detail="Organization cannot participate as
volunteer",
                )

            if existing_application is None:
                if accepted_count >= event.volunteers_needed:
                    raise
HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Свободных
мест больше нет")
                existing_application = EventApplication(
                    event_id=event.id,
                    volunteer_id=profile.id,
                    status=EventApplicationStatus.accepted,
                )
                db.add(existing_application)
            else:
                if (
                    existing_application.status !=
EventApplicationStatus.accepted
                    and accepted_count >= event.volunteers_needed
                ):
                    raise
HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Свободных
мест больше нет")
                existing_application.status =
EventApplicationStatus.accepted

            db.commit()
            db.refresh(event)
            response = self._response(db, event, user)
            response.message = "Вы участвуете как волонтёр"
            return response

        if existing_application is not None:
            response = self._response(db, event, user)
            if existing_application.status ==
EventApplicationStatus.accepted:
                response.message = "Вы уже участвуете"
            elif existing_application.status ==
EventApplicationStatus.rejected:
                response.message = "Ваша заявка была отклонена"
            else:
                response.message = "Заявка уже отправлена"
```

```
            return response

        if accepted_count >= event.volunteers_needed:
            raise HTTPException(status_code=status.HTTP_409_CONFLICT,
detail="Свободных мест больше нет")

        application = EventApplication(
            event_id=event.id,
            volunteer_id=profile.id,
            status=EventApplicationStatus.pending,
        )
        db.add(application)
        db.flush()

        applicant_name = self._profile_display_name(profile)
        event_title = event.title.strip() or "событие"
        notification_service.create(
            db,
            event.creator_id,
            f"{applicant_name} хочет участвовать в событии
«{event_title}».",
            sender_name=applicant_name,
            event_id=event.id,
            application_id=application.id,
        )

        db.commit()
        db.refresh(event)
        response = self._response(db, event, user)
        response.message = "Заявка отправлена"
        return response

    def accept_application(self, db: Session, application_id: int, user:
User) -> EventResponse:
        return self._review_application(
            db,
            application_id=application_id,
            user=user,
            new_status=EventApplicationStatus.accepted,
        )

    def reject_application(self, db: Session, application_id: int, user:
User) -> EventResponse:
        return self._review_application(
            db,
            application_id=application_id,
            user=user,
            new_status=EventApplicationStatus.rejected,
        )

    def get_attendance(self, db: Session, event_id: str, user: User) ->
list[AttendanceItemResponse]:
        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        if event.creator_id != user.id:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can view attendance")

        normalized_status = (event.status or "").strip().lower()
        if normalized_status not in ("attendance", "grouping"):
```

```
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in attendance phase")

        attendance_records = db.scalars(
            select(EventAttendance).where(EventAttendance.event_id ==
event_id)
        ).all()

        return [
            AttendanceItemResponse(
                application_id=record.application_id,
                is_present=record.is_present,
                profile=record.application.volunteer,
            )
            for record in attendance_records
        ]

    def confirm_attendance(
        self,
        db: Session,
        event_id: str,
        user: User,
        present_application_ids: list[int],
    ) -> EventResponse:
        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        if event.creator_id != user.id:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can confirm attendance")

        normalized_status = (event.status or "").strip().lower()
        if normalized_status != "attendance":
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in attendance phase")

        present_set = set(present_application_ids)
        attendance_records = db.scalars(
            select(EventAttendance).where(EventAttendance.event_id ==
event_id)
        ).all()

        now = datetime.now(UTC)
        for record in attendance_records:
            record.is_present = record.application_id in present_set
            record.checked_at = now
            if record.application_id not in present_set:
                application = db.get(EventApplication,
record.application_id)
                if application is not None:
                    self._adjust_rating(db, application.volunteer, -50,
event_id=event_id, event_title=event.title)

        present_count = len(present_set)
        event.present_count = present_count

        if present_count >= 2:
            event.status = "grouping"
            event.group_count = None
            db.commit()
            db.refresh(event)
            response = self._response(db, event, user)
```

```
            response.message = "Выберите количество групп"
        else:
            event.status = "active"
            db.commit()
            db.refresh(event)
            response = self._response(db, event, user)
            response.message = "Событие началось"


    def get_ratable_profiles(self, db: Session, event_id: str, user:
User) -> list[RatableProfileResponse]:
        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        if (event.status or "").strip().lower() != "rating":
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in rating phase")

        profile = self._profile_for_user(db, user)
        is_organizer = event.creator_id == user.id

        # Build attendance map: volunteer_id → is_present
        all_applications = db.scalars(
            select(EventApplication).where(EventApplication.event_id ==
event_id)
        ).all()
        app_by_id = {a.id: a for a in all_applications}
        attendance_records = db.scalars(
            select(EventAttendance).where(EventAttendance.event_id ==
event_id)
        ).all()
        presence_map: dict[int, bool] = {}
        for record in attendance_records:
            app = app_by_id.get(record.application_id)
            if app:
                presence_map[app.volunteer_id] = record.is_present

        absent_volunteer_ids = {vid for vid, present in
presence_map.items() if not present}

        groups = self._get_event_groups(db, event_id)
        has_groups = len(groups) > 0

        profiles: list[Profile] = []
        leader_profile_ids: set[int] = set()

        if is_organizer:
            seen_ids: set[int] = {profile.id}
            if has_groups:
                # Organizer rates group leaders + absent participants
                leader_ids = {g.leader_id for g in groups if g.leader_id
is not None}
                leader_profile_ids = leader_ids
                for lid in leader_ids:
                    p = db.get(Profile, lid)
                    if p and p.id not in seen_ids:
                        seen_ids.add(p.id)
                        profiles.append(p)
            else:
                # No groups — organizer rates all accepted participants
                for app in all_applications:
```

```
                    if app.status == EventApplicationStatus.accepted and
app.volunteer_id not in seen_ids:
                        seen_ids.add(app.volunteer_id)
                        profiles.append(app.volunteer)

            # Always add absent participants (present=False)
            for vid in absent_volunteer_ids:
                if vid not in seen_ids:
                    p = db.get(Profile, vid)
                    if p:
                        seen_ids.add(vid)
                        profiles.append(p)
        else:
            # Leader rates members of own group; regular member has no
one to rate
            group_member = db.scalar(
                select(EventGroupMember).where(
                    EventGroupMember.profile_id == profile.id,
                ).join(EventGroup, EventGroupMember.group_id ==
EventGroup.id).where(
                    EventGroup.event_id == event_id
                )
            )
            if group_member is None:
                return []
            group = db.get(EventGroup, group_member.group_id)
            if group is None or group.leader_id != profile.id:
                return []
            member_rows = db.scalars(
                select(EventGroupMember).where(
                    EventGroupMember.group_id == group.id,
                    EventGroupMember.profile_id != profile.id,
                )
            ).all()
            profiles = [db.get(Profile, m.profile_id) for m in
member_rows]
            profiles = [p for p in profiles if p is not None]

        # Exclude already rated
        already_rated = {
            row.rated_id
            for row in db.scalars(
                select(EventRating).where(
                    EventRating.event_id == event_id,
                    EventRating.rater_id == profile.id,
                )
            ).all()
        }

        return [
            RatableProfileResponse(
                profile_id=p.id,
                first_name=p.first_name,
                last_name=p.last_name,
                avatar_url=p.avatar_url,
                current_rating=p.rating,
                is_present=presence_map.get(p.id, True),
                is_leader=p.id in leader_profile_ids,
            )
            for p in profiles
            if p.id not in already_rated
        ]
```

```
    def submit_ratings(self, db: Session, event_id: str, user: User,
ratings: list[RatingItem]) -> EventResponse:
        event = self.get_event(db, event_id)
        if event is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

        if (event.status or "").strip().lower() != "rating":
            raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in rating phase")

        profile = self._profile_for_user(db, user)
        is_organizer = event.creator_id == user.id

        for item in ratings:
            if not (-50 <= item.score <= 200):
                raise
HTTPException(status_code=status.HTTP_400_BAD_REQUEST, detail="Score out
of allowed range")
            rated_profile = db.get(Profile, item.profile_id)
            if rated_profile is None:
                continue
            existing = db.scalar(
                select(EventRating).where(
                    EventRating.event_id == event_id,
                    EventRating.rater_id == profile.id,
                    EventRating.rated_id == item.profile_id,
                )
            )
            if existing is not None:
                continue
            rating_record = EventRating(
                event_id=event_id,
                rater_id=profile.id,
                rated_id=item.profile_id,
                score=item.score,
            )
            db.add(rating_record)
            self._adjust_rating(db, rated_profile, item.score,
event_id=event_id, event_title=event.title)

        if is_organizer:
            event.status = "completed"
            db.commit()
            db.refresh(event)
            response = self._response(db, event, user)
            response.message = "Оценки сохранены. Событие завершено."
        else:
            db.commit()
            db.refresh(event)
            response = self._response(db, event, user)
            response.message = "Оценки сохранены"

        return response

    def _adjust_rating(
        self,
        db: Session,
        profile: Profile,
        delta: int,
        event_id: str | None = None,
        event_title: str | None = None,
    ) -> None:
        profile.rating = max(0, profile.rating + delta)
```

```
        db.add(RatingHistory(
            profile_id=profile.id,
            delta=delta,
            event_id=event_id,
            event_title=event_title,
            created_at=datetime.now(UTC),
        ))

    def _review_application(
        self,
        db: Session,
        *,
        application_id: int,
        user: User,
        new_status: EventApplicationStatus,
    ) -> EventResponse:
        application = db.get(EventApplication, application_id)
        if application is None:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Application not found")

        event = application.event
        if event.creator_id != user.id:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can review application")

        if new_status == EventApplicationStatus.accepted:
            accepted_count = self._accepted_count(db, event.id)
            if application.status != EventApplicationStatus.accepted and
accepted_count >= event.volunteers_needed:
                raise
HTTPException(status_code=status.HTTP_409_CONFLICT, detail="Свободных
мест больше нет")

        application.status = new_status
        applicant_name =
self._profile_display_name(application.volunteer)
        event_title = event.title.strip() or "событие"

        if new_status == EventApplicationStatus.accepted:
            user_message = f"Ваша заявка на событие «{event_title}»
принята."
            response_message = "Заявка принята"
        else:
            user_message = f"Ваша заявка на событие «{event_title}»
отклонена."
            response_message = "Заявка отклонена"

        notification_service.create(
            db,
            application.volunteer.user_id,
            user_message,
            event_id=event.id,
            application_id=application.id,
        )

        db.commit()
        db.refresh(event)
        response = self._response(db, event, user)
        response.message = f"{response_message}: {applicant_name}"
        return response
```

EventGroupChatService.py

```
def save_group_count_draft(self, db: Session, event_id: str, user: User,
group_count: int) -> EventResponse:
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if event.creator_id != user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can update grouping")

    if (event.status or "").strip().lower() != "grouping":
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in grouping phase")

    event.group_count = group_count
    db.commit()
    db.refresh(event)
    return self._response(db, event, user)

def confirm_grouping(self, db: Session, event_id: str, user: User,
group_count: int) -> EventResponse:
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if event.creator_id != user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can confirm grouping")

    if (event.status or "").strip().lower() != "grouping":
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in grouping phase")

    event.group_count = group_count

    if group_count == 0:
        event.status = "active"
        db.commit()
        db.refresh(event)
        response = self._response(db, event, user)
        response.message = "Событие началось"
        return response

    existing_groups = db.scalars(
        select(EventGroup).where(EventGroup.event_id == event_id)
    ).all()
    existing_numbers = {g.group_number for g in existing_groups}

    for n in range(1, group_count + 1):
        if n not in existing_numbers:
            db.add(EventGroup(event_id=event_id, group_number=n))

    for group in existing_groups:
        if group.group_number > group_count:
            db.delete(group)

    db.commit()
    db.refresh(event)
    response = self._response(db, event, user)
    response.message = "Распределите участников по группам"
    return response
```

```
def auto_distribute(self, db: Session, event_id: str, user: User) ->
list[GroupResponse]:
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if event.creator_id != user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can auto-distribute")

    if (event.status or "").strip().lower() not in _GROUP_EDIT_STATUSES:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in grouping or active phase")

    groups = db.scalars(
        select(EventGroup).where(EventGroup.event_id ==
event_id).order_by(EventGroup.group_number)
    ).all()

    if not groups:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="No groups exist")

    attendance_records = db.scalars(
        select(EventAttendance).where(
            EventAttendance.event_id == event_id,
            EventAttendance.is_present == true(),
        )
    ).all()

    profiles = sorted(
        [record.application.volunteer for record in attendance_records],
        key=lambda p: (
            (p.first_name or p.organization_name or "").lower(),
            (p.last_name or "").lower(),
        ),
    )

    for group in groups:
        group.leader_id = None
        for member in list(group.members):
            db.delete(member)
    db.flush()

    group_count = len(groups)
    for i, profile in enumerate(profiles):
        group = groups[i % group_count]
        if group.leader_id is None:
            group.leader_id = profile.id
        else:
            db.add(EventGroupMember(group_id=group.id,
profile_id=profile.id, role="member"))

    db.commit()
    for group in groups:
        db.refresh(group)

    return [self._group_response(group) for group in groups]

def add_group(self, db: Session, event_id: str, user: User) ->
GroupResponse:
    event = self.get_event(db, event_id)
```

```
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if event.creator_id != user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can add groups")

    if (event.status or "").strip().lower() not in _GROUP_EDIT_STATUSES:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in grouping or active phase")

    max_number = db.scalar(

select(func.max(EventGroup.group_number)).where(EventGroup.event_id ==
event_id)
    ) or 0

    new_group = EventGroup(event_id=event_id, group_number=max_number +
1)
    db.add(new_group)
    db.commit()
    db.refresh(new_group)
    return self._group_response(new_group)

def delete_group(self, db: Session, event_id: str, group_number: int,
user: User) -> None:

def confirm_groups(self, db: Session, event_id: str, user: User) ->
EventResponse:
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if event.creator_id != user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can confirm groups")

    if (event.status or "").strip().lower() != "grouping":
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in grouping phase")

    groups = db.scalars(
        select(EventGroup).where(EventGroup.event_id ==
event_id).order_by(EventGroup.group_number)
    ).all()

    for group in groups:
        if group.leader_id is None:
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail=f"Группа {group.group_number} не имеет лидера",
            )

    event.status = "active"
    db.commit()
    db.refresh(event)
    response = self._response(db, event, user)
    response.message = "Событие началось"
    return response

def get_chats(self, db: Session, event_id: str, user: User) ->
list[ChatRoomResponse]:
```

```
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if (event.status or "").strip().lower() != "active":
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Chat is only available for active events")

    groups = self._get_event_groups(db, event_id)
    is_organizer = event.creator_id == user.id
    profile = db.scalar(select(Profile).where(Profile.user_id ==
user.id))

    if not groups or len(groups) == 1:
        if not self._can_access_chat(db, event, user):
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Access denied")
        return [ChatRoomResponse(id=0, event_id=event_id, type="single",
title="Чат события")]

    # Multiple groups
    if is_organizer:
        return [ChatRoomResponse(id=LEADERS_CHAT_ID, event_id=event_id,
type="leaders", title="Чат лидеров")]

    if profile is None:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Access denied")

    user_group = None
    user_is_leader = False
    for group in groups:
        if group.leader_id == profile.id:
            user_group = group
            user_is_leader = True
            break
        if any(m.profile_id == profile.id for m in group.members):
            user_group = group
            break

    if user_group is None:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Access denied")

    leader_chat_title = "Чат с организатором" if user_is_leader else
"Чат лидеров"
    result = [
        ChatRoomResponse(
            id=user_group.group_number,
            event_id=event_id,
            type="group",
            group_number=user_group.group_number,
            title=f"Чат группы {user_group.group_number}",
        )
    ]
    if user_is_leader:
        result.append(ChatRoomResponse(id=LEADERS_CHAT_ID,
event_id=event_id, type="leaders", title=leader_chat_title))

    return result

def send_chat_message(
    self,
```

```
    db: Session,
    event_id: str,
    chat_id: int,
    user: User,
    content: str | None,
    photo_url: str | None,
) -> MessageResponse:
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")

    if (event.status or "").strip().lower() != "active":
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Chat is only available for active events")

    self._validate_chat_access(db, event, chat_id, user)

    profile = db.scalar(select(Profile).where(Profile.user_id ==
user.id))
    if profile is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Profile required")

    message = EventMessage(
        event_id=event_id,
        sender_profile_id=profile.id,
        chat_id=chat_id,
        content=content,
        photo_url=photo_url,
    )
    db.add(message)
    db.commit()
    db.refresh(message)

    groups = self._get_event_groups(db, event_id)
    organizer_profile_id = self._organizer_profile_id(db, event)
    return self._build_message_response(message, organizer_profile_id,
groups, chat_id)

def _get_grouping_event_and_group(
    self, db: Session, event_id: str, group_number: int, user: User
) -> tuple[Event, EventGroup]:
    event = self.get_event(db, event_id)
    if event is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Event not found")
    if event.creator_id != user.id:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
detail="Only organizer can modify groups")
    if (event.status or "").strip().lower() not in _GROUP_EDIT_STATUSES:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Event is not in grouping or active phase")
    group = db.scalar(
        select(EventGroup).where(
            EventGroup.event_id == event_id,
            EventGroup.group_number == group_number,
        )
    )
    if group is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
detail="Group not found")
    return event, group
```

```
@staticmethod
def _validate_profile_is_present(db: Session, event_id: str, profile_id:
int) -> None:
    attendance = db.scalar(
        select(EventAttendance)
        .join(EventApplication, EventAttendance.application_id ==
EventApplication.id)
        .where(
            EventAttendance.event_id == event_id,
            EventAttendance.is_present == true(),
            EventApplication.volunteer_id == profile_id,
        )
    )
    if attendance is None:
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Person is not a present attendee")

@staticmethod
def _remove_profile_from_all_groups(

def _group_response(group: EventGroup) -> GroupResponse:
    return GroupResponse(
        id=group.id,
        event_id=group.event_id,
        group_number=group.group_number,
        leader_id=group.leader_id,
        leader=group.leader,
        members=[
            GroupMemberResponse(
                profile_id=member.profile_id,
                role=member.role,
                profile=member.profile,
            )
            for member in group.members
        ],
    )

@staticmethod
def _accepted_count(db: Session, event_id: str) -> int:
```